

# **BLACKJACK**

## **A CARD GAME**

**Ireoluwa Dairo**

**CSC/CIS 17C -42513**

**SPRING 2025**

**Project 2**

# INTRODUCTION

This project is a modified version of a C++ Blackjack game that makes use of the Standard Template Library (STL), trees, recursions, recursive sort and hashing. I chose Blackjack because it's a well-known card game with clear rules. The game follows classic Blackjack rules: get as close to 21 as possible without going over.

At the start of each round, both player and dealer receive two cards. Players can "hit" (draw cards), "stand" (keep hand), "double down" (double bet for one card), or request strategy "hints" from an advisor. The dealer follows standard casino rules, hitting until reaching 17 or higher. The game determines winners by comparing final totals without busting.

Github - [CSC\\_17C/project at main · Kvngjaid04/CSC\\_17C](#)

## Summary

This project has over 1200 lines of code and 40+ functions, using several C++ concepts:

- **STL Containers:** vector, list, map, set, stack, and queue
- **STL Algorithms:** for\_each, transform, count\_if, generate, random\_shuffle, find
- **Iterators:** used throughout the code
- **Lambda functions:** paired with STL algorithms for cleaner logic
- **Binary Tree Operations:** Tree rotations, balancing, and strategy lookup algorithms
- **Recursive Functions:** Player turn logic with depth tracking, double-down validation
- **Recursive Sorting:** Merge sort implementation for deck display functionality
- **Hash Functions:** djb2 primary hash with secondary hash for collision resolution
- **Tree Algorithms:** AVL rotations (LL, RR, LR, RL) for maintaining balance

**Hash Table Implementation:** The custom Hash class uses double hashing to store player profiles. When players log in, the hash table quickly retrieves their data using the djb2 hash

function with collision resolution. This allows multiple players to save/load their progress efficiently.

**AVL Tree for Strategy (hints):** The AVL tree stores Blackjack strategy based on player total and dealer upcard.

**Recursive Design:** Player turns use recursive function calls with depth parameters. The double-down feature uses recursion with depth tracking to validate betting logic. Merge sort recursively sorts the deck for display purposes.

**Why Graphs Weren't Used:** I couldn't find or think of a logical application within Blackjack's framework. The game is more about randomness and probability rather than connected relationships or pathfinding.

## **Description**

The game features a player management system that uses hash tables for quick data retrieval and storage. The AVL tree serves as a strategy advisor, providing optimal move recommendations based on current hand values and dealer cards.

New players begin with \$1000 starting cash and each round follows a structured flow: players place bets with stack-based history tracking, receive cards stored in vector containers, and make strategic decisions guided by the advisor. The recursive player turn system manages complex game states. After each game, players can view statistics including card frequency analysis through maps, unique card tracking via sets, and complete action history maintained in queues. Player progress persists through file operations integrated with hash table management for efficient data handling. The merge sort algorithm provides organized deck viewing capabilities. Gameplay continues until players choose to quit or deplete their funds.

# Sample Input/Output

The screenshot shows a C++ IDE with a project named 'Blackjack\_v3'. The source code is displayed in the center, and the output window on the right shows the initial game state.

```
1 //
2 * Author: Ireoluwa
3 * Created: October 29, 2024
4 * Modified for CSC 17 C
5 * Purpose: Blackjack Game
6 */
7
8 // System Libraries
9 #include <iostream>
10 #include <cstdlib>
11 #include <ctime>
12 #include <vector>
13 #include <list>
14 #include <map>
15 #include <set>
16 #include <stack>
17 #include <queue>
18
19 // User Libraries
20 #include "Card.h"
21 #include "Save.h"
22 #include "Deck.h"
23 #include "Play.h"
24 #include "Menu.h"
25 #include "UserMgr.h"
26 #include "AVL.h"
27 // #include "Graph.h"
28 #include "Hash.h"
29
30 using namespace std;
31
32 // Function prototypes for main game flow
33 void setup(Save&, string&, map<int, int>&, set<Card>&,
34           queue<string>&, AVL&);
35 void round(vector<Card>&, int&, list<Card>&, list<Card>&,
36           int&, int&, Save&, string&, map<int, int>&,
37           set<Card>&, queue<string>&, stack<int>&, AVL&);
38 bool endRnd(Save&, list<Card>&, list<Card>&, int&, map<int, int>&,
39            set<Card>&, queue<string>&);
40 void reset(list<Card>&, list<Card>&, int&, queue<string>&);
41 bool askRul(bool&);
42
43 // Main execution
44 int main(int argc, char** argv) {
45     srand(time(0));
46
47     // Game containers
48     vector<Card> deck(52);
```

The output window shows the following text:

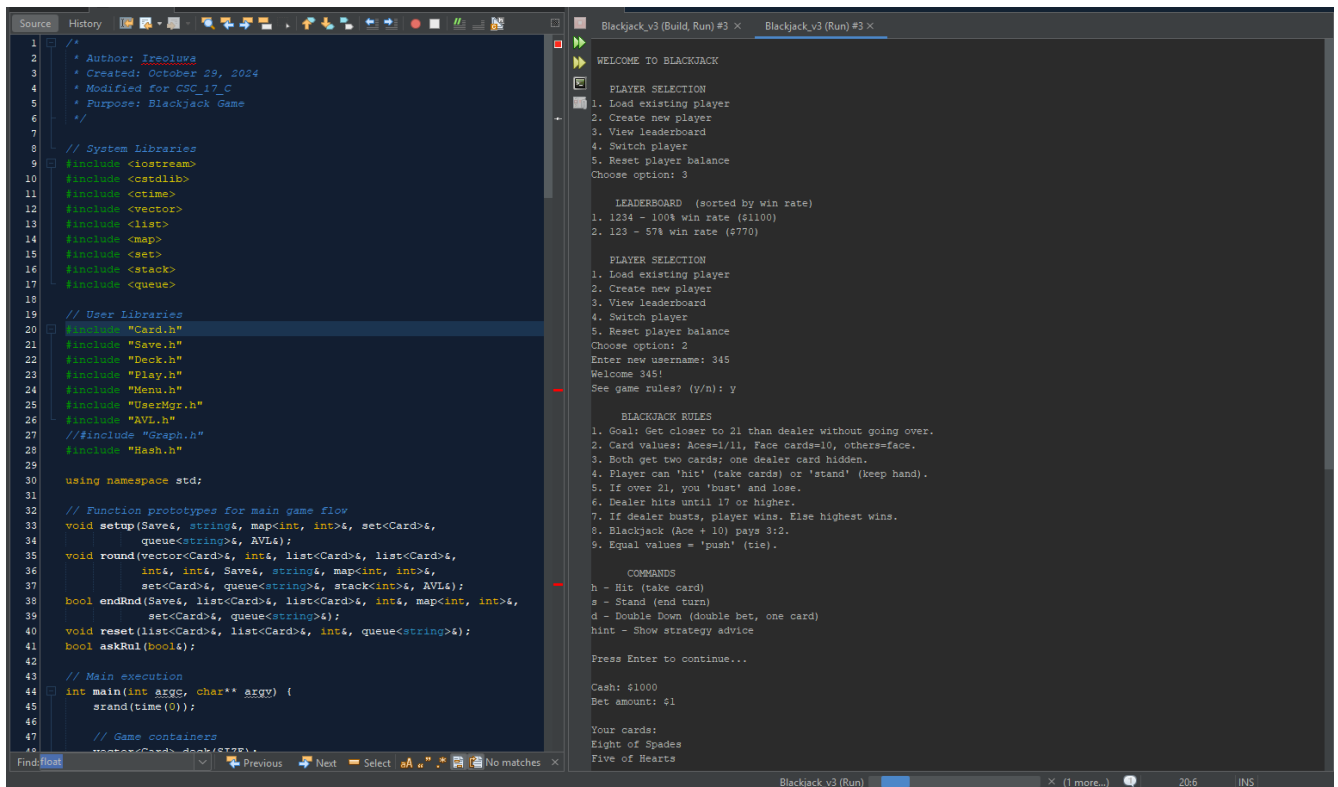
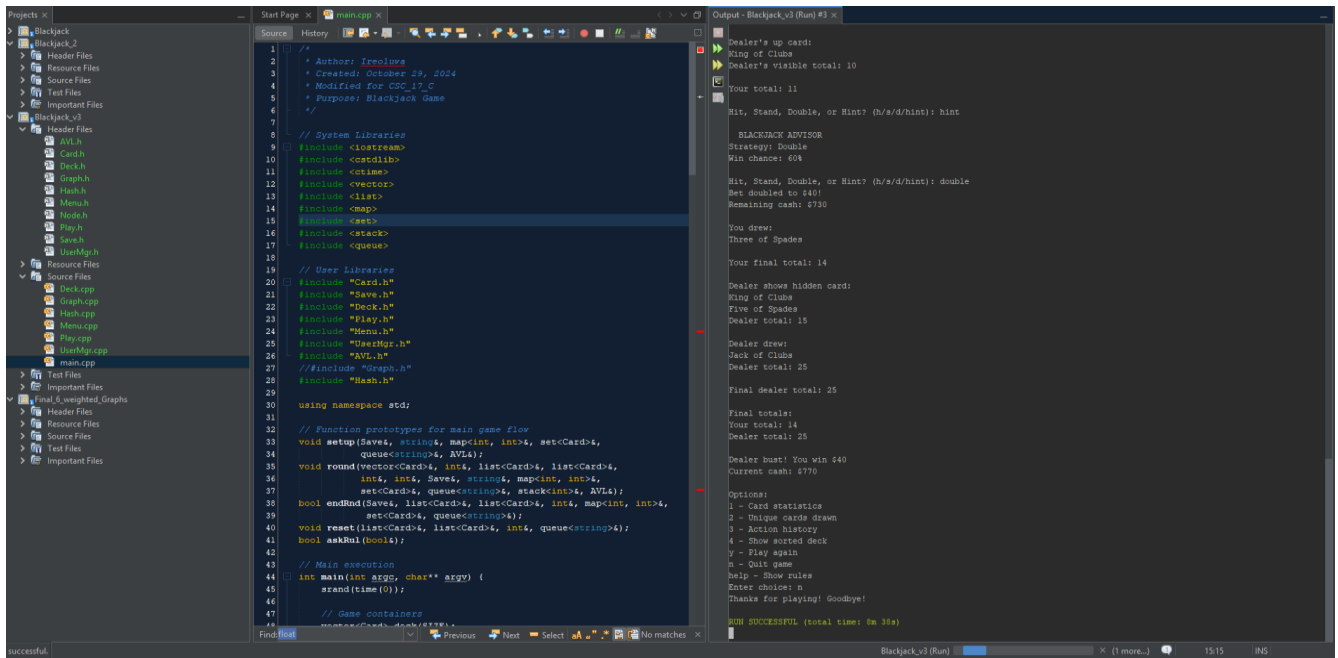
```
WELCOME TO BLACKJACK
PLAYER SELECTION
1. Load existing player
2. Create new player
3. View leaderboard
4. Switch player
5. Reset player balance
Choose option: 1
Available players:
1. 123 (4 games, 50% win rate, 50)
2. 1234 (1 games, 100% win rate, $1100)
Select player: 1
Player 123 has 50! Reset balance to $1000? (y/n): y
Balance reset to $1000!
Welcome back 123!
See game rules? (y/n): n
Cash: $1000
Bet amount: $500
Your cards:
Jack of Clubs
Jack of Diamonds
Dealer's up card:
Nine of Clubs
Dealer's visible total: 9
Your total: 20
Hit, Stand, Double, or Hint? (h/s/d/hint): hint
BLACKJACK ADVISOR
Strategy: Stand
Win chance: 55%
Hit, Stand, Double, or Hint? (h/s/d/hint): s
Dealer shows hidden card:
Nine of Clubs
Ten of Spades
Dealer total: 19
Final dealer total: 19
Final totals:
Your total: 20
Dealer total: 19
```

The screenshot shows the same C++ IDE with the source code of the Blackjack game. The output window on the right shows the continuation of the game.

```
1 //
2 * Author: Ireoluwa
3 * Created: October 29, 2024
4 * Modified for CSC 17 C
5 * Purpose: Blackjack Game
6 */
7
8 // System Libraries
9 #include <iostream>
10 #include <cstdlib>
11 #include <ctime>
12 #include <vector>
13 #include <list>
14 #include <map>
15 #include <set>
16 #include <stack>
17 #include <queue>
18
19 // User Libraries
20 #include "Card.h"
21 #include "Save.h"
22 #include "Deck.h"
23 #include "Play.h"
24 #include "Menu.h"
25 #include "UserMgr.h"
26 #include "AVL.h"
27 // #include "Graph.h"
28 #include "Hash.h"
29
30 using namespace std;
31
32 // Function prototypes for main game flow
33 void setup(Save&, string&, map<int, int>&, set<Card>&,
34           queue<string>&, AVL&);
35 void round(vector<Card>&, int&, list<Card>&, list<Card>&,
36           int&, int&, Save&, string&, map<int, int>&,
37           set<Card>&, queue<string>&, stack<int>&, AVL&);
38 bool endRnd(Save&, list<Card>&, list<Card>&, int&, map<int, int>&,
39            set<Card>&, queue<string>&);
40 void reset(list<Card>&, list<Card>&, int&, queue<string>&);
41 bool askRul(bool&);
42
43 // Main execution
44 int main(int argc, char** argv) {
45     srand(time(0));
46
47     // Game containers
48     vector<Card> deck(52);
```

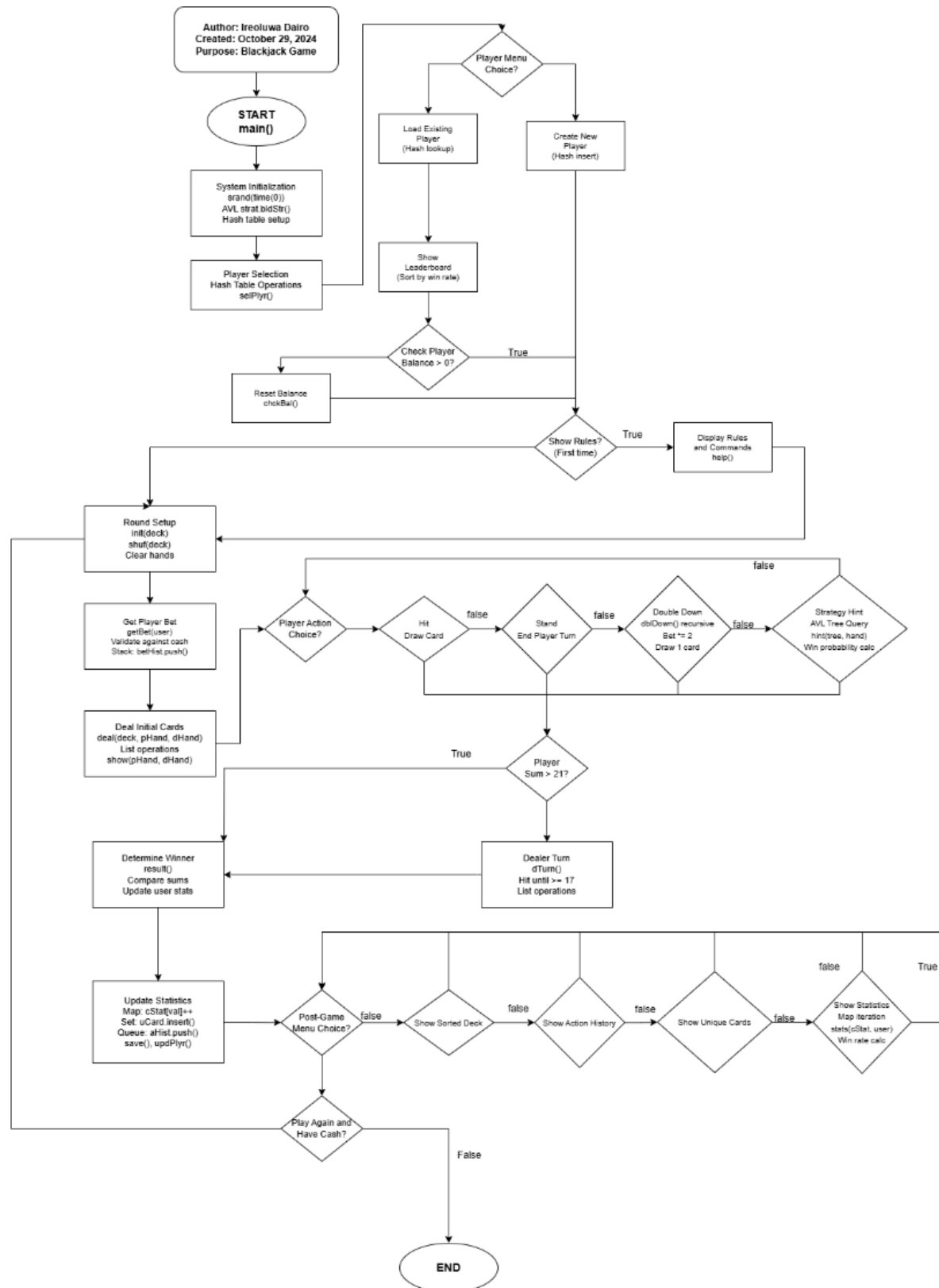
The output window shows the following text:

```
4 - Show sorted deck
y - Play again
n - Quit game
help - Show rules
Enter choice: y
Cash: $1500
Bet amount: $750
Your cards:
Ten of Spades
Queen of Clubs
Dealer's up card:
Seven of Hearts
Dealer's visible total: 7
Your total: 20
Hit, Stand, Double, or Hint? (h/s/d/hint): hint
BLACKJACK ADVISOR
Strategy: Stand
Win chance: 75%
Hit, Stand, Double, or Hint? (h/s/d/hint): s
Dealer shows hidden card:
Seven of Hearts
Six of Diamonds
Dealer total: 13
Dealer drew:
Three of Diamonds
Dealer total: 16
Dealer drew:
Five of Hearts
Dealer total: 21
Final dealer total: 21
Final totals:
Your total: 20
Dealer total: 21
You lose $750
Current cash: $750
Options:
1 - Card statistics
2 - Unique cards drawn
3 - Action history
4 - Show sorted deck
```



# FLOWCHART

Note: This flowchart represents the general logic of the code, not the exact execution flow.



## Pseudocode

Start program

Initialize random number generator using `srand(time(0))`

Create STL containers:

- `vector<Card>` deck for the card deck
- `list<Card>` phand, dhand for player and dealer hands
- `map<int, int>` cstat for card statistics
- `set<Card>` ucard for tracking unique cards
- `queue<string>` ahist for recording game actions
- `stack<int>` bethist for betting history
- Hash allplyr for player data storage
- AVL strat for strategy hints

Initialize AVL tree with blackjack strategy using `strat.bldstr()`

Load existing player files into hash table using `lodexst()`

Call `selplyr()` function to select or create player using hash operations

Repeat the game loop:

- Initialize the deck using `init(deck, ncards)` with vector operations
- Shuffle deck using `shuf(deck)` with STL `random_shuffle` algorithm
- Ask for rules display if first time using `askrul()`
- Get the player's bet using `getbet()` and validate against `user.cash`
- Push bet to betting history stack using `bethist.push(bet)`
- Clear player and dealer hands using `phand.clear()`, `dhand.clear()`
- Deal initial cards using `deal()` function, adding to lists with `push_back()`
- Display the initial cards using `show()` function
- Record "Initial deal" action in history queue using `ahist.push()`

Play the player's turn using `pturn()` recursively:

- Calculate hand value using `sumhand()` with STL `for_each` algorithm
- Display current total and options (hit/stand/double/hint)
- If hit requested, draw card and add to player's list using `push_back()`
- If double requested, call `dbldown()` recursive function with depth parameter
- If hint requested, query AVL tree using `strat.gethint()` for strategy advice
- Update hand sum and check for bust condition
- Continue recursively until stand or bust

Record "Player turn complete" in action history using `ahist.push()`

If player has not busted (`psum <= 21`), play dealer's turn:

- Call `dturn()` function using list iterators
- Hit until dealer sum  $\geq 17$  using while loop

- Display dealer cards using list iteration

Record "Dealer turn complete" in action history using `ahist.push()`

Handle the outcome using `result()` function:

- Compare final totals
- Update `user.wins`, `user.games`, `user.cash`
- Display winner and cash remaining

Save the game state using `save(user, fname)` to binary file

Update player in hash table using `updplyr(user)`

Update card statistics using STL containers:

- Iterate through player hand with range-based for loop
- Update map using `cstat[val]++` for each card value
- Insert cards into set using `ucard.insert()` for unique tracking

Display post-game menu using `getmenu()`:

- Show card statistics using `stats()` with map iteration and `for_each`
- Report unique cards count from set container using `ucard.size()`
- Display action history from queue using temporary queue and `front()/pop()`
- Show sorted deck using merge sort algorithm `srtdeck()` and `mrg()`
- Allow player to choose next action

Check if player can continue using `canplay()`:

- Verify `user.cash > 0`
- If broke, display game over message

If playing again:

- Clear all containers using `reset()` function
- Clear hands with `phand.clear()`, `dhand.clear()`
- Empty action history queue using `while(!Ahist.empty()) ahist.pop()`
- Reset deck size to `SIZE` constant
- Continue to next iteration

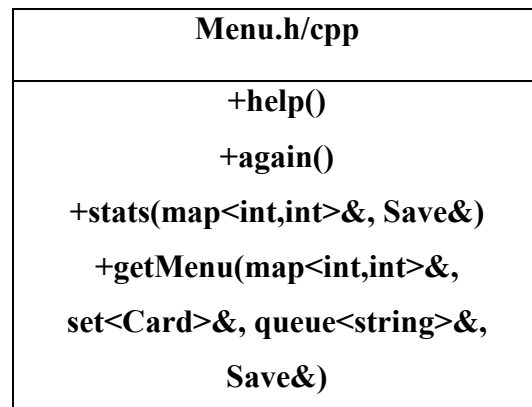
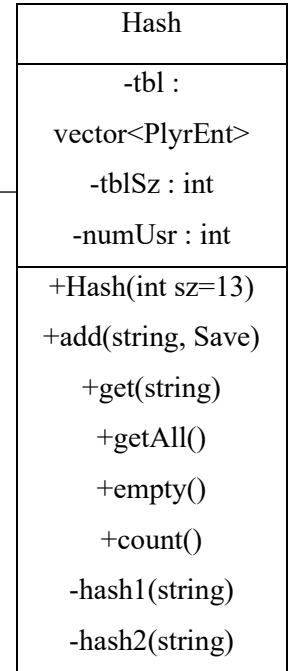
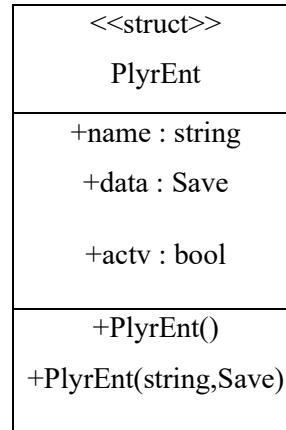
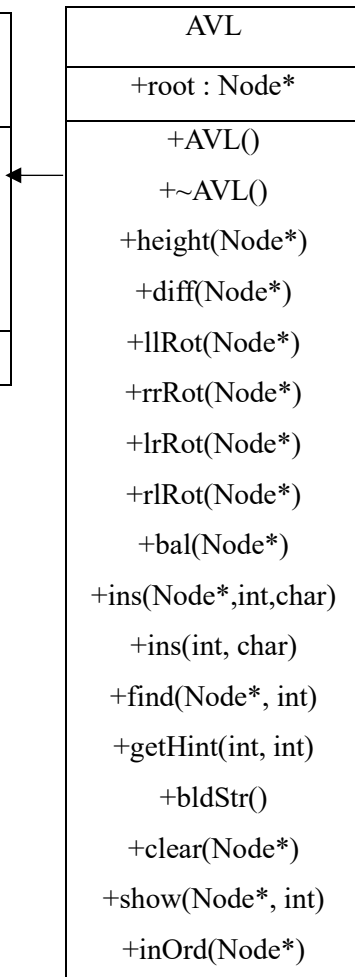
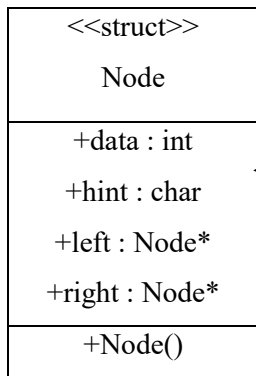
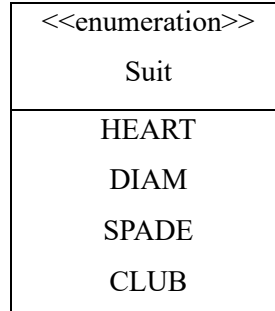
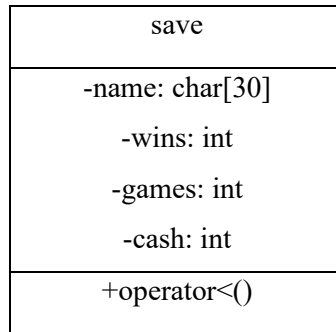
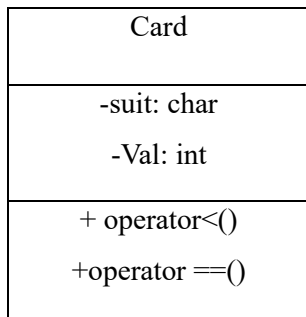
End game loop when player quits or runs out of money

Display final goodbye message

Clean up all STL containers automatically via destructors

End program

## UML



| Play.h/cpp                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> +sumHand(list&lt;Card&gt;&amp;) +dblDown(Save&amp;, int&amp;,           int=0) +deal(vector&lt;Card&gt;&amp;, list&lt;Card&gt;&amp;,list&lt;Card&gt;&amp;) +show(list&lt;Card&gt;&amp;, list&lt;Card&gt;&amp;, Vector&lt;Card&gt;&amp;) +hint(AVL&amp;, list&lt;Card&gt;&amp;,       int, int) +pTurn(vector&lt;Card&gt;&amp;, list&lt;Card&gt;&amp;, int&amp;, Save&amp;, int&amp;,       AVL&amp;) +dTurn(vector&lt;Card&gt;&amp;,       list&lt;Card&gt;&amp;, int&amp;) +result(Save&amp;, int, int, int) +canPlay(Save&amp;) </pre> |

| Usermgr.h/cpp                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> +load(Save&amp;, string) +save(Save, string) +valname(string) +clrbuf() +getbet(Save&amp;) +calcrte(int, int) +lodexst() +valdata(Save&amp;) +selplyr() +updplyr(Save&amp;) +lodexst(vector&lt; plyrent&gt;&amp;) +creatnw() +shwlead(vector&lt; plyrent&gt;&amp;) +rsetbal(vector&lt; plyrent&gt;&amp;) +chckbal(Save&amp;) </pre> |

| STL Containers in main()                                                                                                                                                                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> Vector&lt;Card&gt; deck(SIZE) - Card deck storage List&lt;Card&gt; phand, dhand - Player/dealer hands Map&lt;int, int&gt; cstat - Card statistics tracking Set&lt;Card&gt; ucard - Unique cards drawn Queue&lt;string&gt; ahist - Action history log Stack&lt;int&gt; bethist - Betting history stack </pre> |

| Deck.h/cpp                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> +init(vector&lt;Card&gt;&amp;, int&amp;) +shuf(vector&lt;Card&gt;&amp;) +draw(vector&lt;Card&gt;&amp;) +getSuit(int) +getVal(int) +pCard(int, char) +srtDeck(vector&lt;Card&gt;&amp;,int, int) +mrg(vector&lt;Card&gt;&amp;, int, int, int) +showDck(vector&lt;Card&gt;&amp;) </pre> |

## CHECKOFF SHEET

1) Container classes (Where in code did you put these Concepts and how were they used?)

1. Sequences (At least 1)

a. **List:**

**main.cpp (ln 49-50) - list<Card> pHand; list<Card> dHand;**  
**store player/dealer cards**

Play.cpp (ln 63,72) - pHand.push\_back(drawn); - add cards to hand

b. **slist**

c. **bit\_vector:**

main.cpp (ln 48) - vector<Card> deck(SIZE); - main deck storage

Deck.cpp (ln 23) - deck.push\_back(c); - build deck

Deck.cpp (ln 38) - deck.pop\_back(); - draw cards

2. Associative Containers (At least 2)

a. **Set:**

main.cpp (ln58) - set<Card> uCard; - track unique cards drawn

main.cpp (ln 161) - uCard.insert(\*it); - add unique cards

b. **Map:**

main.cpp (ln 57) - map<int, int> cStat; - card statistics counter

main.cpp (ln 160) - cStat[val]++; - increment card count

Deck.cpp (ln 54) - map<int, string> vluNmcs - card name lookup

c. **Hash**

3. Container adaptors (At least 2)

a. **Stack:**

main.cpp (ln 60) - stack<int> betHist; - betting history

main.cpp (ln 132) - betHist.push(bet); - record bets

b. **Queue:**

main.cpp (ln 59) - queue<string> aHist; - action history

main.cpp (ln 142) - aHist.push("Initial deal"); - log game actions

c. **priority\_queue**

d. **Iterators**

## 2) Iterators

### 1. Concepts (Describe the iterators utilized for each Container)

a. Trivial Iterator

b. Input Iterator

c. Output Iterator

d. Forward Iterator:

Menu.cpp (ln 79) - stat.begin(), stat.end() - traverse map for statistics

e. Bidirectional Iterator:

Play.cpp (ln 179) - for (auto it = hand.begin(); it != hand.end(); ++it)  
- traverse card lists

f. Random Access Iterator:

Deck.cpp (ln 76) - deck.begin() + lft - direct position access for merge sort

## 3) Algorithms (Choose at least 1 from each category)

### 1. Non-mutating algorithms

a. **for\_each:**

Play.cpp (ln 37) - for\_each(hand.begin(), hand.end(), [&](const Card& card) - calculate hand sum

Menu.cpp (ln 79) - for\_each(stat.begin(), stat.end(), [](const pair<int, int>& item) - display stats

b. find

c. count:

UserMgr.cpp (ln 50) - count\_if(name.begin(), name.end(), [](char c)  
- validate username chars

d. equal

e. search

### 2. Mutating algorithms

a. copy

b. Swap

c. Transform:

Menu.cpp (ln 110) - transform(input.begin(), input.end(),  
input.begin(), ::tolower); - convert to lowercase

- d. Replace
- e. fill
- f. Remove
- g. Random\_Shuffle:

Deck.cpp (ln 31) - random\_shuffle(deck.begin(), deck.end()); -  
shuffle deck

### 3. Organization

- a. Sort:

UserMgr.cpp (ln 208) - sort(players.begin(), players.end(), [](const  
PlyrEnt& a, const PlyrEnt& b) - sort leaderboard

- b. Binary search

- c. Merge:

Deck.cpp (ln 96) - srtDeck() - custom merge sort implementation

- d. inplace\_merge
- e. Minimum and maximum

## Variables and Data Structures

### STL Containers:

**main.cpp (ln 48-50,57-60) - deck, hands, stats, history containers**

Structures with Custom Operator:

Card.h (ln 17,24) - operator< and operator== for set/map usage

Save.h (ln 21) - operator< for player comparison

### Concepts used

- Containers: vector, list, set, map, stack, queue
- Algorithms: for\_each, count\_if, transform, random\_shuffle, sort
- Iterators: bidirectional (list), random access (vector), forward (set/map)
- Lambda expressions: hand calculation, validation, sorting, display

## REFERENCES

1. Originally developed for CSC-5, enhanced for CSC-17A, and now updated with STL implementation, hashing, trees, recessions and recursive sort for CSC-17C by Ireoluwa Dairo
2. C++ STL Documentation [Standard Template Library Programmer's Guide](#)
3. Blackjack rules: <https://www.bicyclecards.com/how-to-play/blackjack/>